

Ingegneria del Software

Introduzione

Prof. Paolo Giorgini

A.A. 2026-2027

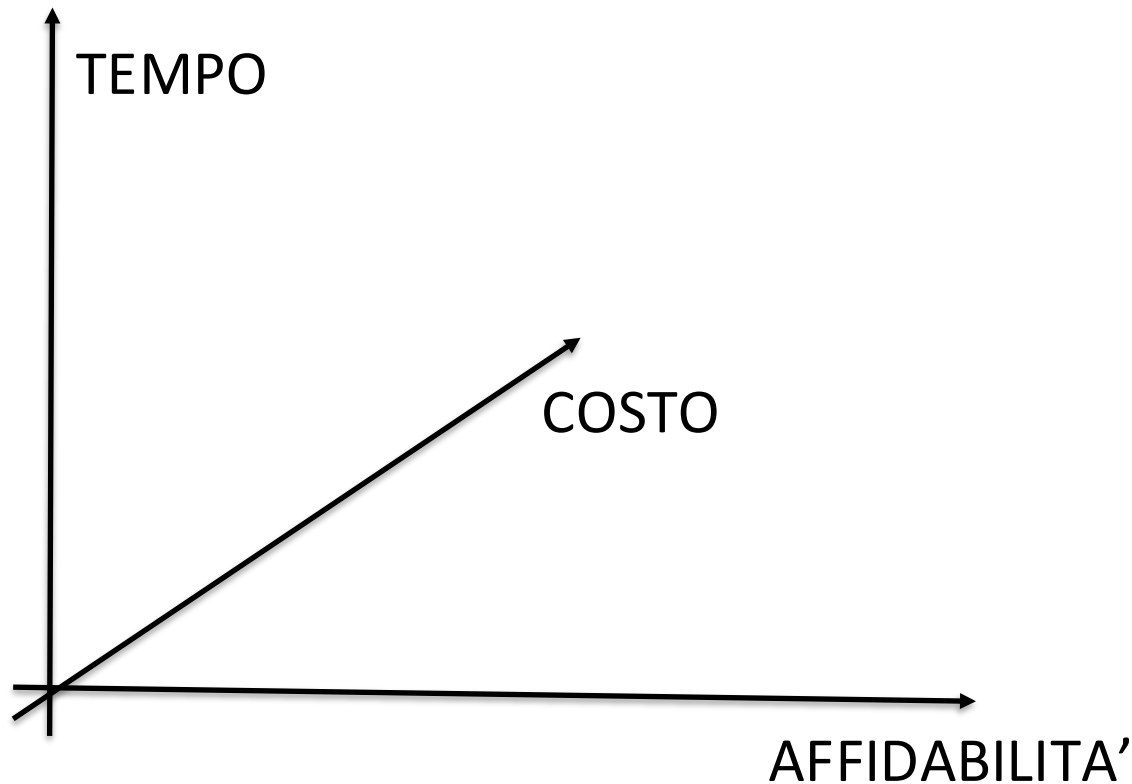
Oggi

- Inquadramento del corso
- Presentazione Comune e BTS
- Organizzazione del corso
- Progetto di Esame
- Materiale e supporto
- Challenge
- Q/A

Software

- Software sempre più indispensabile
 - servizi, sanità, PA, commercio, finanza, industria, intrattenimento
 - Pensate all'impatto dell'IA su tutti i settori produttivi
- Disponibilità del software
 - Venduto con un click
 - Fenomeno delle App
- Incorporato in ogni genere di sistema
 - trasporti, macchinari medici, sistemi telecom, sistemi militari, apparecchi industriali, intrattenimento, ufficio, ...

Dimensioni dello sviluppo software

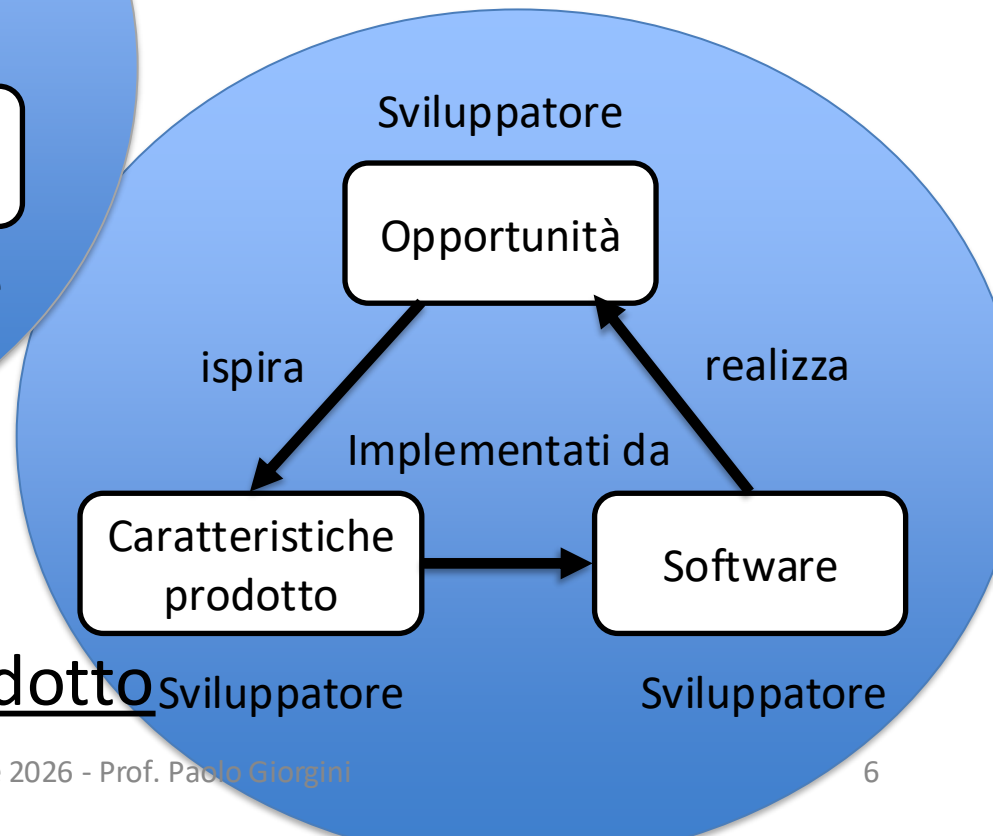
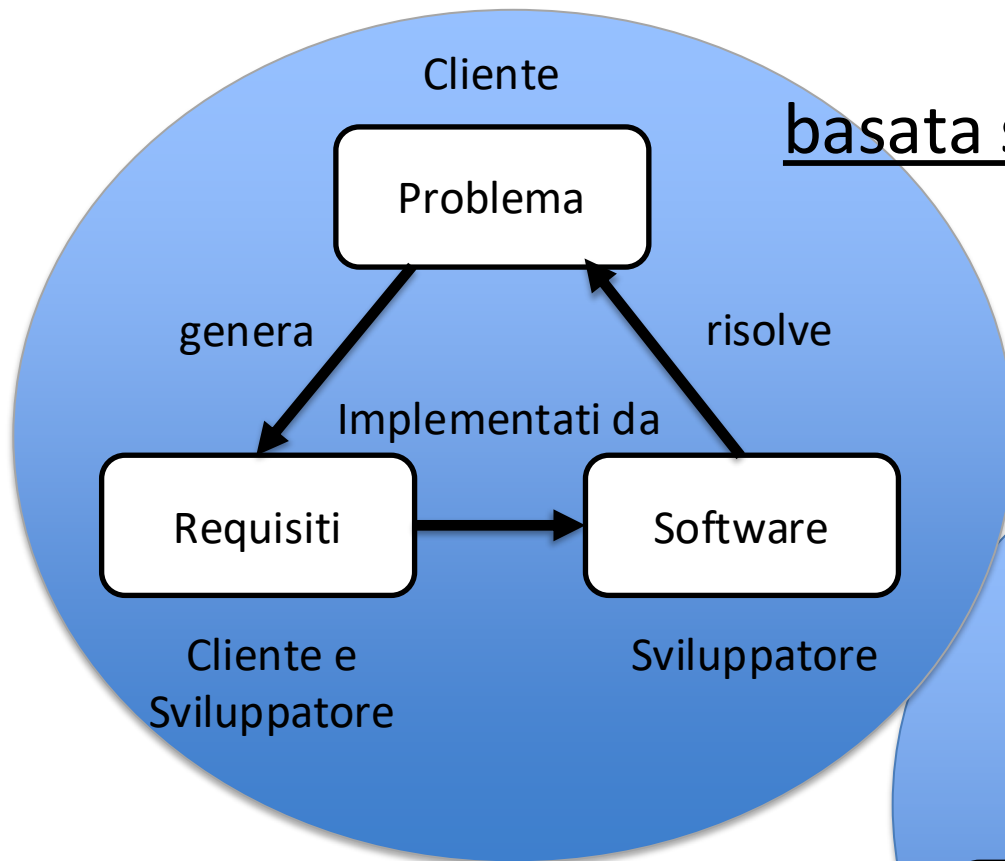


Manutenibilità, estensibilità, integrabilità, portabilità, usabilità, documentazione, ...

Qualità del software

- Software di qualità
 - Parametri di qualità e misure
 - Interni (es. verificabilità, manutenibilità, riusabilità, portabilità ...)
 - Esterni (es. correttezza, affidabilità, robustezza, efficienza, usabilità, scalabilità ...)
- Necessità di strumenti e tecniche per
 - aumentare la qualità e allo stesso tempo semplificare, accelerare e rendere più economico lo sviluppo
- Ingegneria del software
 - Principi, metodi, strumenti, processi

Ingegneria del Software



Who are we?



Tipica interazione cliente-provider



*Vorrei un software che mi
permetta di vendere i miei
prodotti online*

*Deve “collegarsi” al sistema
di gestione del magazzino
e delle vendite*

Mi raccomando, la sicurezza!!

Versione web e mobile

Lo vorrei pronto a fine mese



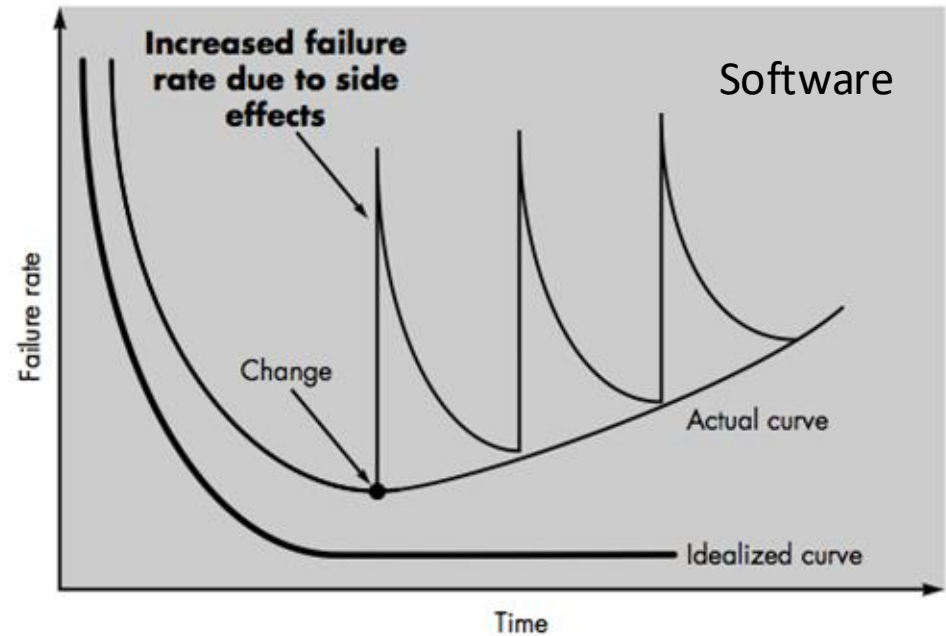
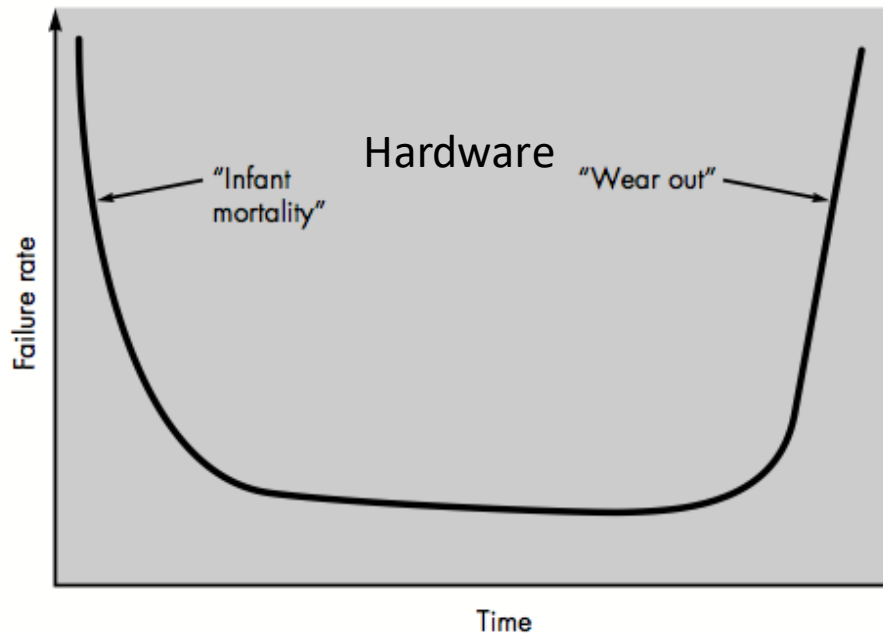
Prodotto intangibile

- Perché tempi così lunghi?
- Perché costi così elevati?
- Perché non sono stati individuati tutti gli errori prima della consegna?
- Perché così tanto tempo e impegno nel mantenimento di un software esistente?
- Perché è difficile misurare i progressi dello sviluppo del software?

Ingegneria del software - risposte a queste domande attraverso metodi sistematici e tecniche consolidate nel tempo

Caratteristiche del software

- Si sviluppa o si struttura, non si fabbrica nel senso tradizionale
 - Sviluppo software $\neq$ fabbricazione hardware



Il software non si consuma

Software Evolution

Cambiamento continuo (1974): gli E-system devono

Crescita continua (1980): il contenuto funzionale degli E-

Riduzione della qualità (1996): gli utenti percepiscono la qualità degli E-system come in **costante declino**, a meno che i sistemi non vengano gestiti rigorosamente e adattati ai cambiamenti dell'ambiente operativo

accettazione una decrescita. Pertanto la crescita incrementale media rimane al più costante durante l'evoluzione del sistema

- La legge della crescita continua (1980): il **contenuto funzionale degli E-system deve essere costantemente incrementato** in modo da mantenere la soddisfazione degli utenti nel corso della vita del sistema
- La legge della riduzione della qualità (1996): gli utenti percepiscono la qualità degli E-system come in **costante declino**, a meno che i sistemi non vengano gestiti rigorosamente e adattati ai cambiamenti dell'ambiente operativo
- La legge di feedback del sistema (1996): I processi evolutivi di un E-system costituiscono sistemi di feedback multilivello, multiciclo, multiagente e devono essere trattati come tali per ottenere un miglioramento significativo

Fonte: Lehman, M., et al, "Metrics and Laws of Software Evolution—The Nineties View," *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*, IEEE, 1997, reperibile sul sito

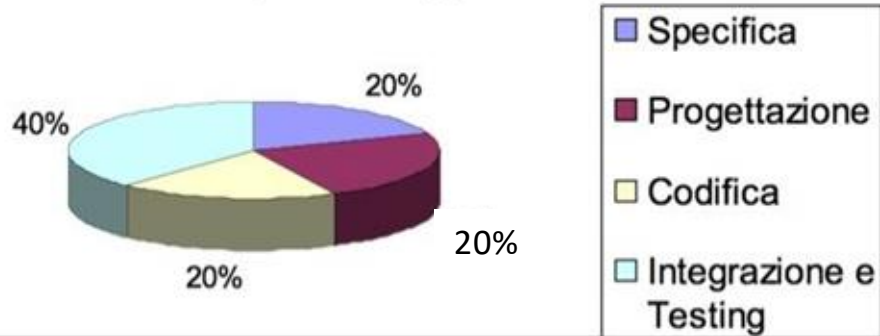
Ma siamo proprio sicuri?

- Management
 - Se siamo in ritardo, possiamo sempre recuperare aumentando il numero di programmatori
 - Se decido di far realizzare un progetto software a una terza parte posso restare tranquillo perché tutto il lavoro sarà svolto esternamente
- Programmatori
 - Scritto il software e messo in opera il nostro lavoro è finito
 - Il solo prodotto di un progetto concluso è il programma funzionante

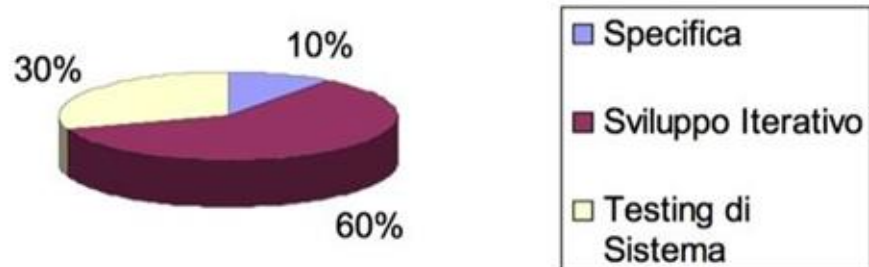
Costi del software

- I costi del Software > Hardware
 - mantenimento/evoluzione > sviluppo
 - variano in base al tipo, le prestazioni o l'affidabilità
- La distribuzione dei costi dipende anche dal tipo di **modello di sviluppo** adottato
 - Casi in cui Il 60% dei costi è speso per le attività di sviluppo, il 40% per il testing

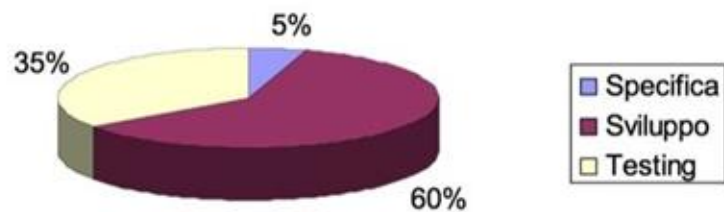
Costi per lo sviluppo Waterfall



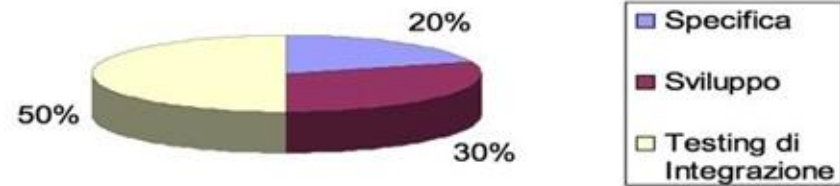
Costi dello Sviluppo Iterativo



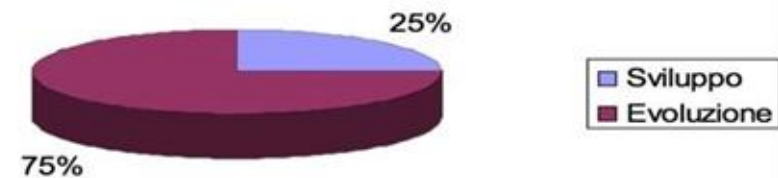
Costi di sviluppo di un prodotto generico



Costi del Component Based Software Engineering



Costi per lo Sviluppo ed evoluzione di sistemi di lunga durata



La qualità del software

- Un software di qualità dovrebbe fornire le funzionalità e le prestazioni richieste ed essere allo stesso tempo
 - **mantenibile, fidato, efficiente, accettabile...**
- I problemi legati alla qualità del software sono difficili da rilevare:
 - molti difetti del software sono introdotti nelle fasi iniziali di specifica e design, ma verranno rilevati molto più avanti, in operatività....

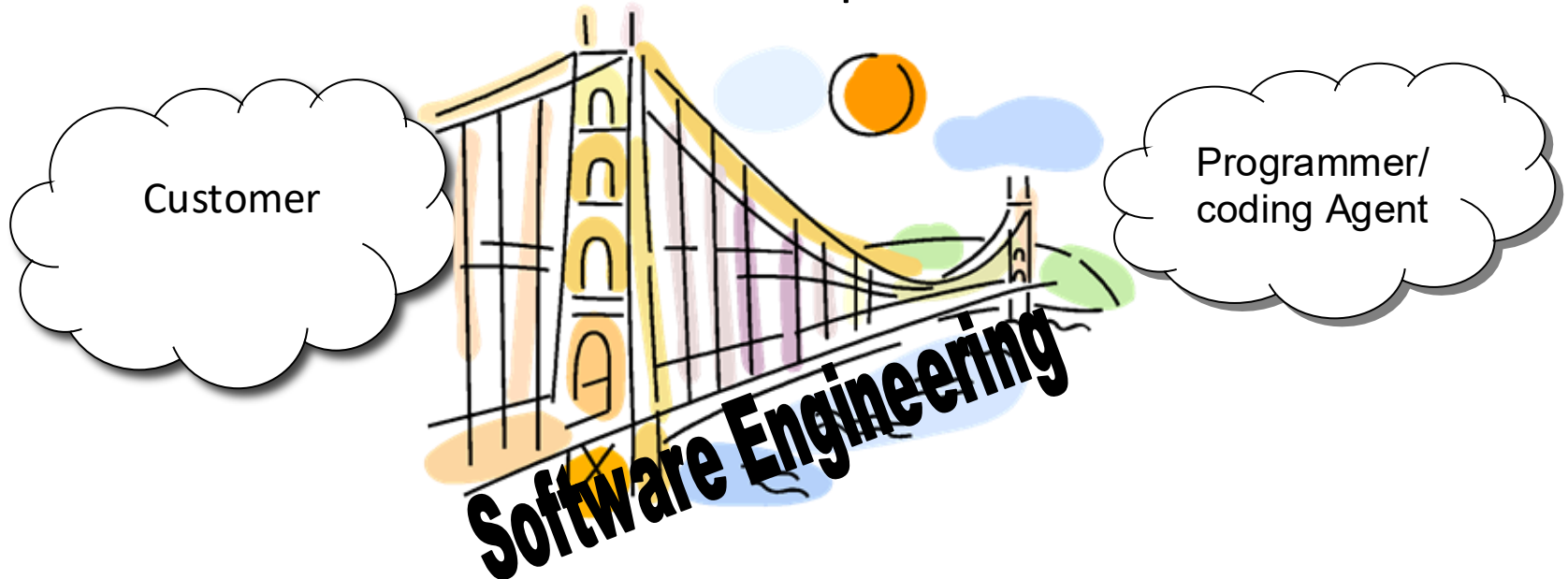
L'Ingegneria del Software come disciplina nasce nel 1968 con la conferenza NATO di Garmisch, come risposta alla crisi del software, e da lì si è evoluta fino ad abbracciare le pratiche moderne che conosciamo oggi

L'ingegneria del software

- E' una disciplina ingegneristica che si occupa di **tutti gli aspetti** relativi allo sviluppo del software
- Gli ingegneri del software:
 - Adottano un **approccio sistematico** e organizzato per il loro lavoro;
 - Usano **strumenti e tecniche** appropriate
 - variabili a seconda del problema da risolvere, dei vincoli di sviluppo, e delle risorse disponibili

Scopo dell'Ingegneria del software

- Risolvere un problema complicato
 - Non ben definito e non ben presentato



Prima regola dell'ingegneria del software

Un problema non può essere delegato — né a un team né a un AI Coding Agent — senza averlo prima capito, specificato e reso verificabile (per raffinamenti)

Vibe Coding vs SE con AI

La differenza non è usare o non usare l'AI: è mantenere **controllo sul processo**

VIBE CODING

**Idea → prompt → codice →
correzioni**

- contesto spesso implicito
- feedback basato su “sembra funzionare”
- modifiche difficili da controllare
- adatto a prototipi ed esplorazione

SOFTWARE ENGINEERING CON AI

**Requirements → spec → plan
→ tasks → code → tests**

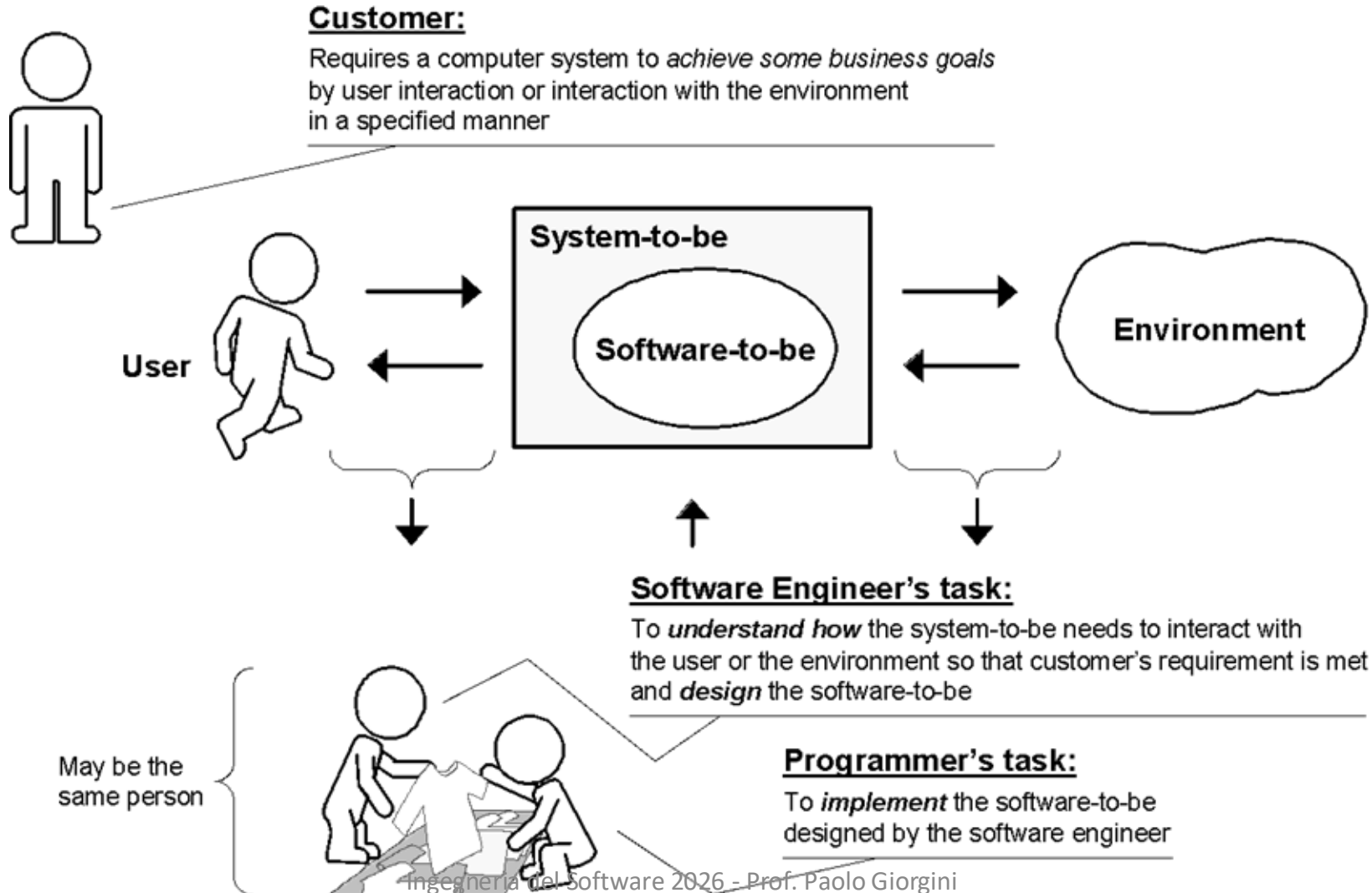
- contesto esplicito e documentato
- acceptance criteria e test
- traceability tra requisiti, codice e verifica
- adatto a sistemi evolutivi e mantenibili

Obiettivo del corso

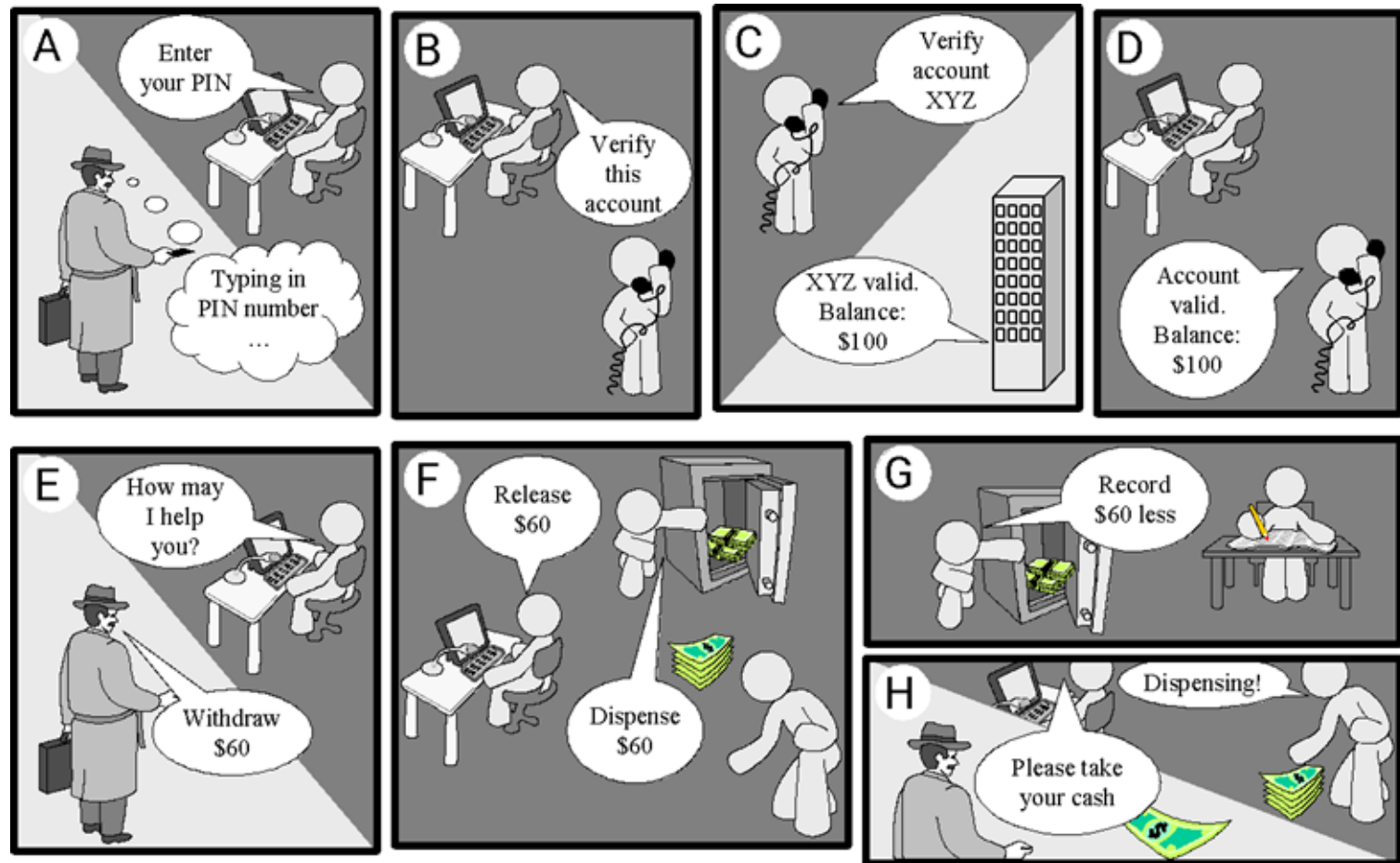
Delegare parte dell'implementazione senza delegare il controllo

Prompt migliori aiutano. Specifiche verificabili e tracciabili cambiano il processo.

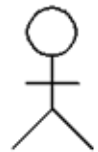
Ruoli dell'Ingegneria del Software



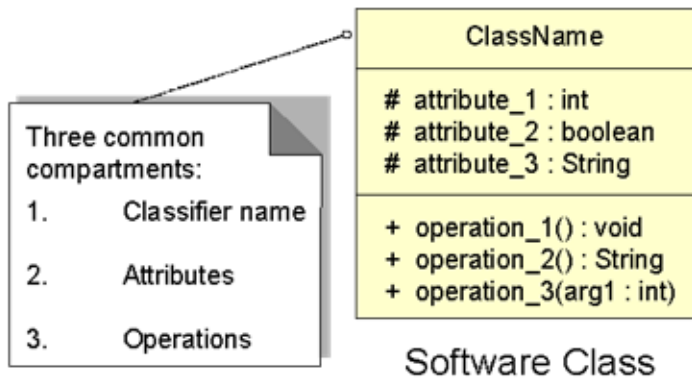
Specificare un problema



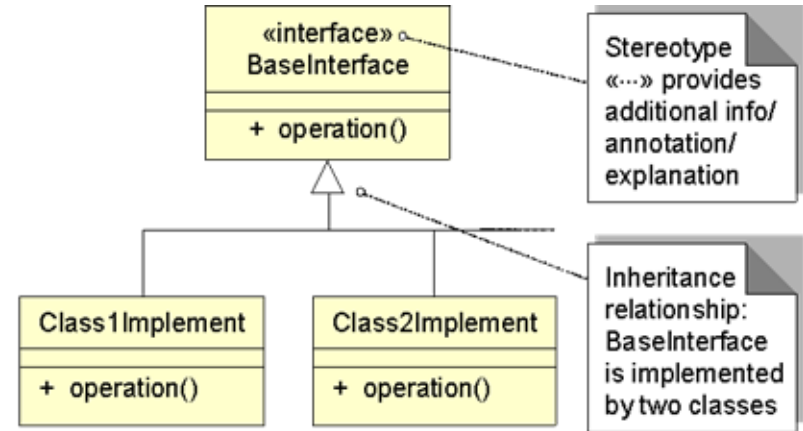
Specifica del problema in UML



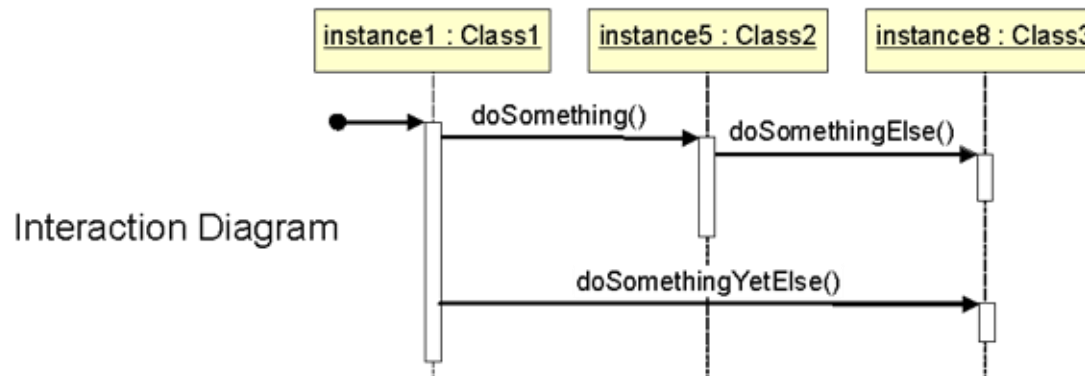
Actor



Comment



Software Interface Implementation

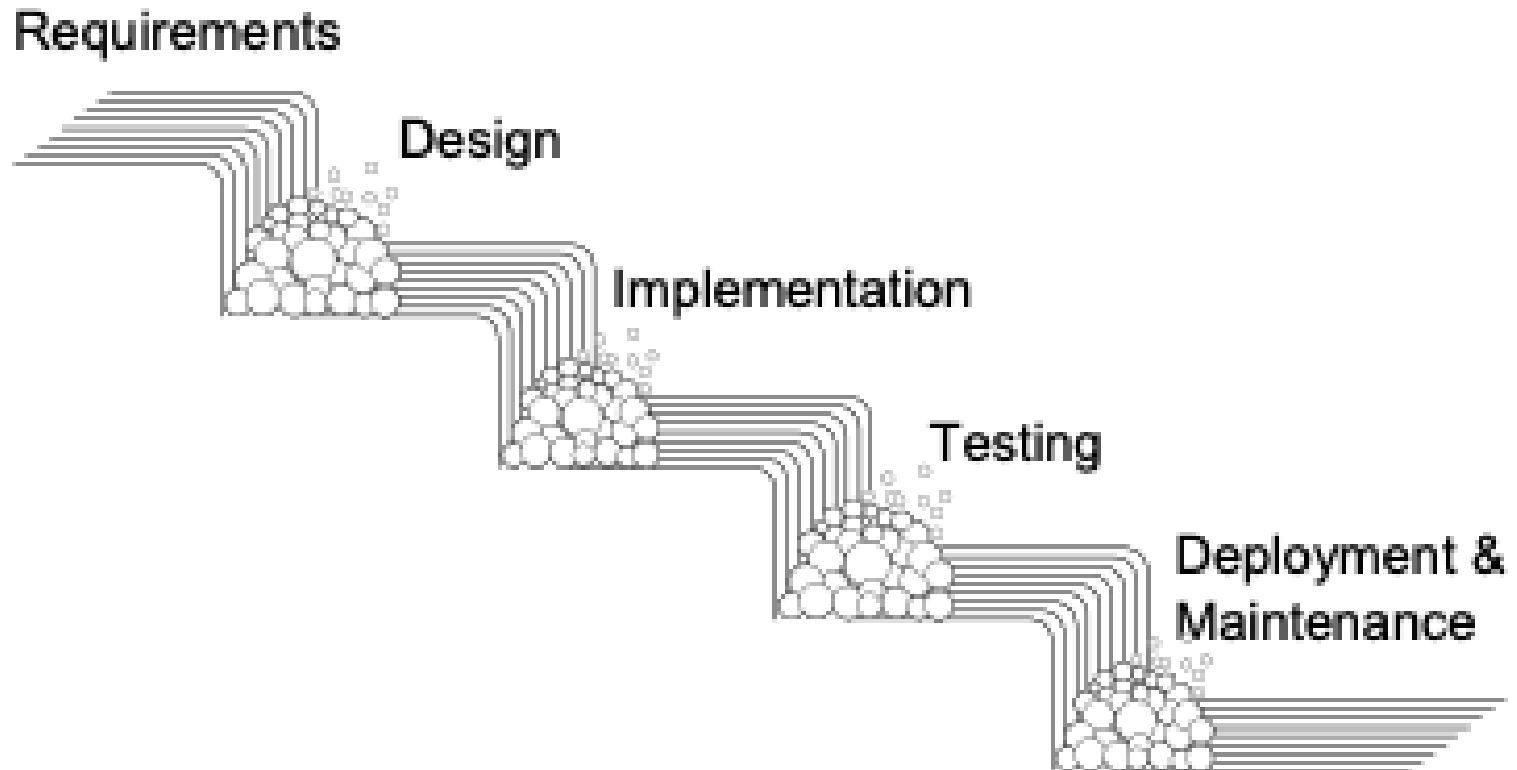


Processi Software

- Un insieme di attività aventi per obiettivo lo sviluppo o l'evoluzione di un sistema software
- Ogni processo tipicamente include:
 - **Specifica** – definizione di ciò che il sistema dovrà fare e dei vincoli di progettazione
 - **Sviluppo** – progettazione e programmazione
 - **Testing e Verifica**
 - **Validazione** – si verifica che il software sia esattamente ciò che il cliente richiede
 - **Evoluzione** – si modifica il software per adeguarlo a requisiti dell'utente e del mercato che cambiano

Modello di processo software

- Descrive il processo: **Waterfall**, Iterative, Incrementale, Component-based....



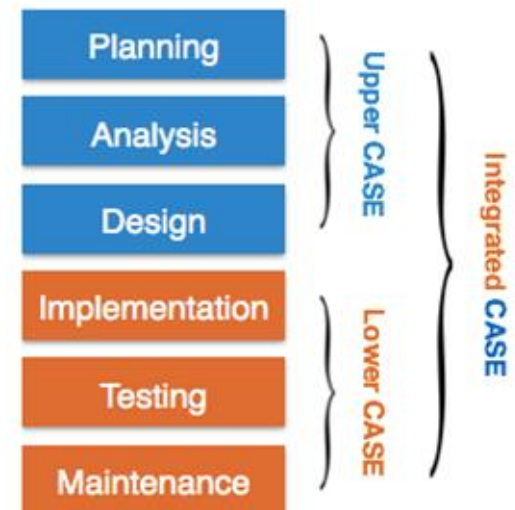
Processo Agile

SCRUM PROCESS



Metodi, strumenti, standard

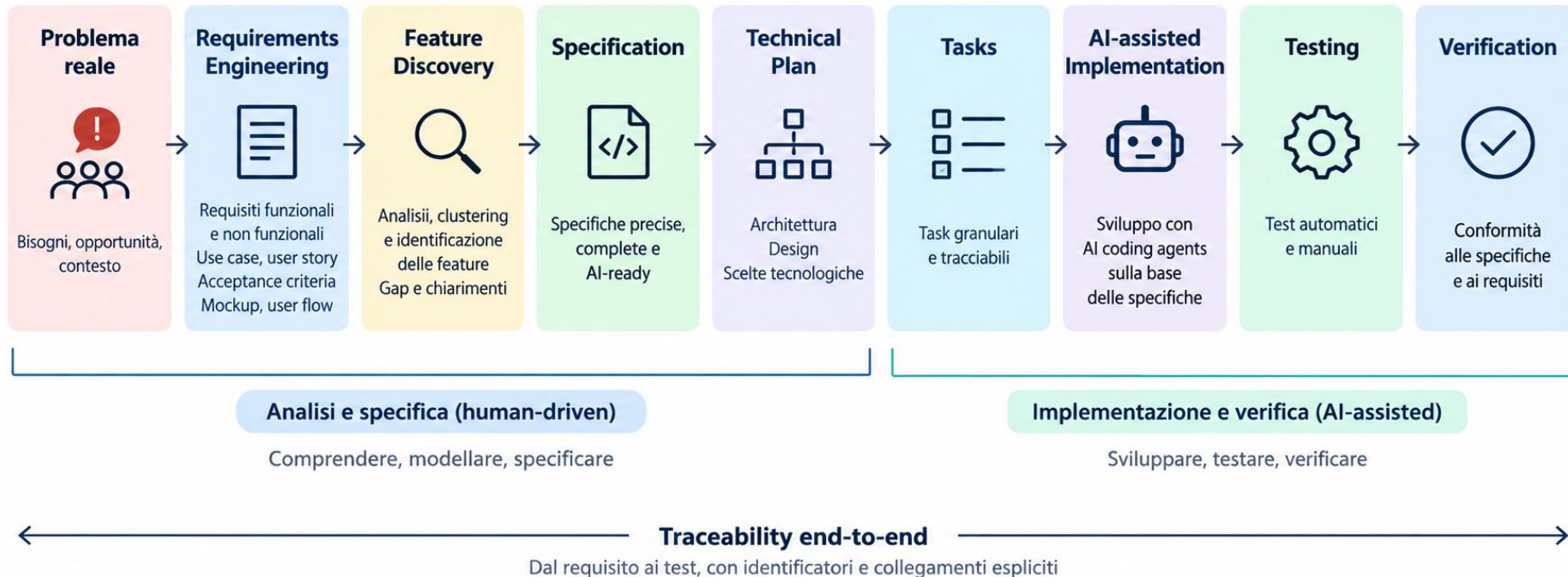
- Metodi: Approcci strutturati per sviluppare software di qualità, a costi contenuti. (Es. SADT, JSD, OOA, OOD, etc..)
 - Es. Specificano i modelli da usare, le regole a cui sottostare, forniscono una guida alle attività dei processi e alla relativa organizzazione.
- Strumenti: Sistemi software usati per aiutare le attività dei processi software (es. analisi, modellazione, debugging, testing)
 - Es. Upper CASE e Lower CASE
- E ancora ...
 - Standard, Normative, Linee Guida, Principi, ... (CMM, CMM-I, ISO 9000, ISO 12207, IEEE std...)



Dal problema reale al software

Un processo guidato da specifiche, potenziato dall'AI

L'AI supporta l'implementazione,
ma l'ingegneria del software guida il processo.



AI Cod Agents: cosa cambia realmente

Gli agenti accelerano l'implementazione solo se ricevono contesto, confini e criteri di verifica

Da “generatore di codice” a “esecutore controllato di task”

Prima

- prompt ad hoc
- contesto incompleto
- output difficile da valutare

Nel corso

- specification AI-ready
- task granulari
- quality gates umani

Risultato atteso

- meno ambiguità
- più controllo
- verifica della conformità

Human gates

Gli studenti decidono se una feature è pronta per passare da requisiti a spec, da spec a piano, da task a implementazione, da test a verification.

L'AI può produrre codice; il team resta responsabile di correttezza, qualità e traceability

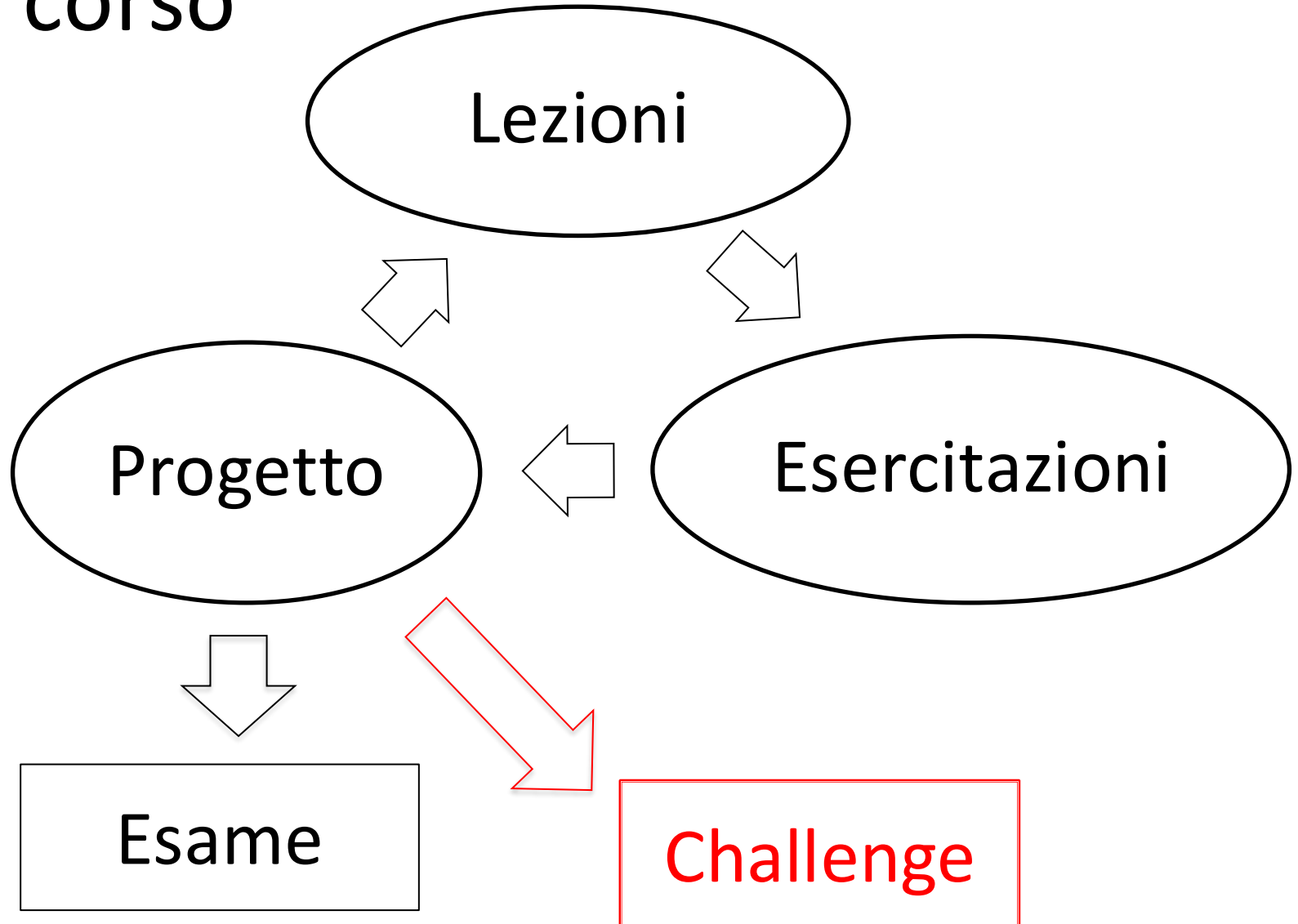
Obiettivo del corso

- Il corso fornisce **principi, metodi e strumenti** dell'Ingegneria del Software per analizzare, specificare, progettare, implementare e verificare sistemi software complessi, anche con il supporto di AI Coding Agents, mantenendo controllo, qualità e tracciabilità
 - Affronteremo tematiche legate all'analisi e alla progettazione di un sistema software complesso esplorando modelli di processo di sviluppo, linguaggi di modellazione e tecniche di verifica e validazione
 - **Svilupperete un progetto completo**

Cosa cambia nel corso a.a.26/27

- Più enfasi su Requirements Engineering e Feature Discovery
- Introduzione dello Spec-Driven Development
- Uso controllato di AI Coding Agents
- Maggiore attenzione a traceability, testing e verification
- Deliverable orientati a specifiche AI-ready

Il corso



Lezioni

- Teoriche, Tutorials, Esercitazioni
- Seminari
 - Aziende ed esperti
- Zoom e registrazioni
 - link zoom per ciascuna lezione su moodle
 - Registrazione delle lezioni su moodle

TODO asap: iscriversi al corso su Moodle

<https://didatticaonline.unitn.it/dol/course/view.php?id=44284>

Progetto

- Analisi, Progettazione e Sviluppo di una WebApp
- Servizio per il Comune di Trento
 - Utilizzo dell'AI
- **A seguire la presentazione**



100 Progetti per il Comune di Trento



UNIVERSITÀ
DI TRENTO



COMUNE
DI TRENTO



TRENTINOSVILUPPO
IMPRESA INNOVAZIONE MARKETING TERRITORIALE



Challenge

“100 progetti per il comune di Trento”



I migliori 3 progetti verranno selezionati e premiati dal Comune – cerimonia



Challenge

“100 progetti per il comune di Trento”



La sfida nella sfida

I migliori tre progetti tra i migliori tre di INF e ICE



Challenge

“100 progetti per il comune di Trento”



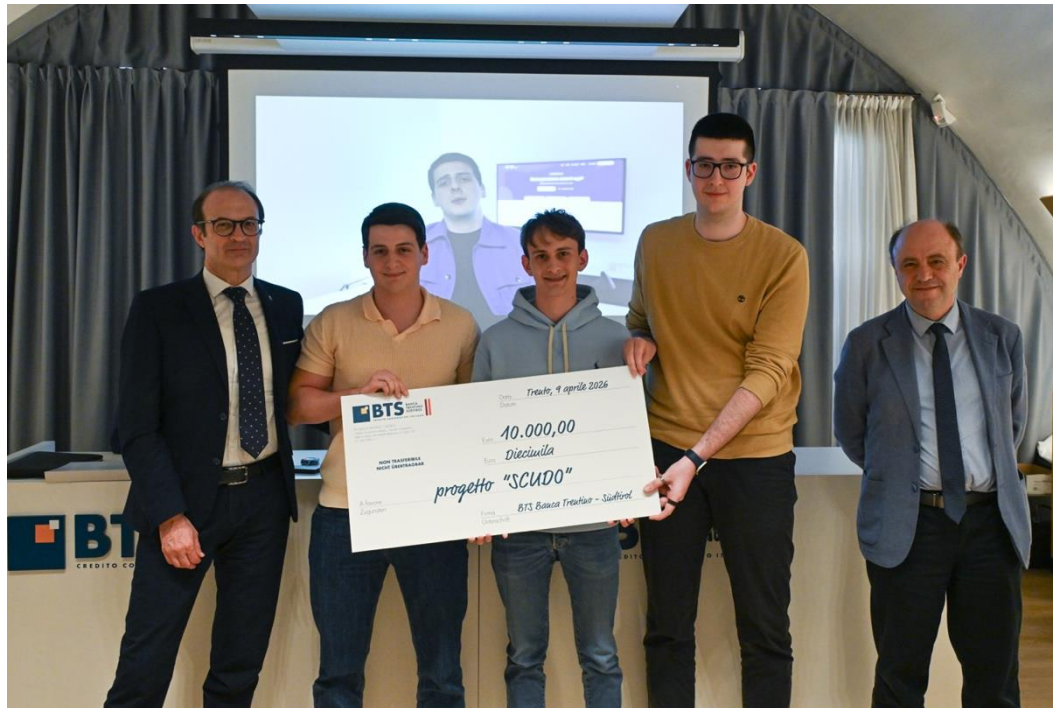
Premio imprenditorialità



Challenge

“100 progetti per il comune di Trento”

Il miglior progetto





- Andrea Schir
 - Responsabile Risorse umane - Parità e Inclusione



- **Giacomo Fioroni**

- Dirigente del Servizio Innovazione digitale, Responsabile per la transizione digitale (RTD) e Responsabile della conservazione

- **Alex Tomasi**

- Funzionario del Servizio Innovazione Transizione Digitale del Comune di Trento

Progetto & Challenge



Gruppi

- Gruppi: 3 studenti
 - Form google da compilare (vedi slide successiva)
 - Ciascun gruppo lavora come un team di progetto
 - Il progetto prevede la consegna di 4 deliverable a scadenze fisse durante il corso:
 - D1 – Requirements Engineering
 - D2 – AI-Ready Software Specification
 - D3 – Technical Plan, Tasks, Implementation & Testing
 - D4 – Report finale, Verification & Spec Compliance

Per formare i gruppi

- Ogni gruppo (3 studenti) deve compilare
 - <https://forms.gle/JaMFN4cxSwufLzAJ9>
 - Scadenza: **giovedì 17 settembre**
- Se siete <3 studenti compilate comunque la scheda (come singolo o coppia), formeremo noi poi i gruppi
- Indicate il team leader (primo membro del gruppo)

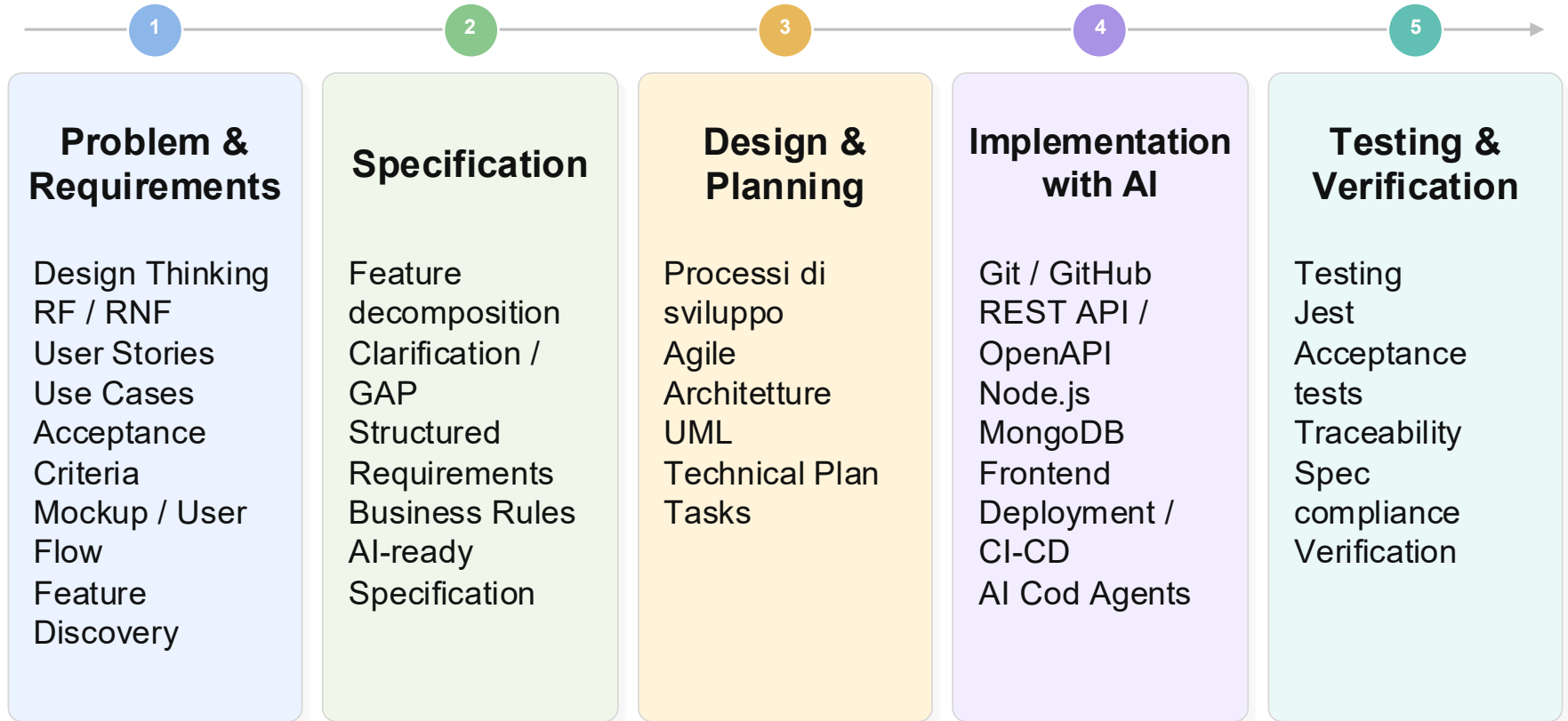
Team leader e Log

- Ogni gruppo ha un **team leader** che avra' il compito di coordinare le attivita', gestire le criticita', organizzare gli incontri del gruppo (deve ovviamente lavorare anche ai deliverable)
- Il carico di lavoro deve essere distribuito in maniera uniforme tra i membri del gruppo
 - Ricordatevi di tener traccia del lavoro svolto in maniera da poter quantificare nel report finale il lavoro svolto da ciascuno

Esame

- **Consegna** dei deliverable e documento finale
 - I deliverable hanno delle scadenze ma possono essere rivisti
- **Criteri di valutazione:** correttezza nell'uso dei linguaggi-metodi-strumenti, qualità analisi-progettazione-sviluppo, lavoro di team

Il percorso del corso



TRACEABILITY & HUMAN CONTROL THROUGHOUT THE PROCESS

Requirements → Feature → Spec → Plan → Task → Code → Test → Verification

Quando e dove

- Lezioni in presenza
 - Lunedì 10.30-13.30 (B107)
 - Martedì 15.30-18.30 (B109)
 - Mercoledì 8.30-10.30 (A101)

Materiale e libri di testo

- Slides e altro materiale disponibile su moodle
- Alcuni libri consigliati
 - **Ingegneria del software: fondamenti e principi (seconda edizione).** Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli, Pearson Printice Hall.
 - **Introduzione all'ingegneria del software moderna.** Ian Sommerville. Pearson Printice Hall.
 - **Ingegneria del software (ottava edizione).** Ian Sommerville. Pearson Printice Hall.
 - **Principi di Ingegneria del Software (quinta edizione).** Roger S. Pressman, MacGraw-Hill.

FAQ

- FAQ: Documento accessibile su moodle
- Form domande / richieste / commenti sempre attiva

La Squadra



Paolo Giorgini



Marco Robol



Alessandro Tomasi



Gabriele Padovani



Tutors

Tutors



- Ad ogni Gruppo verrà assegnato un tutor
 - Seguirà il Progetto fino agli appelli di gennaio/febbraio
 - Incontri periodici e su appuntamento

Domande?

- Se sono da solo/a posso comunque fare il progetto?
- Che succede se un componente del gruppo ci molla durante il corso?
- E' obbligatorio partecipare alla challenge?
- Se faccio l'esame a giugno posso partecipare alla challenge?
- Se vinco la challenge ho un punteggio più alto all'esame?
- Si riesce a fare il progetto per l'appello di gennaio/febbraio?
- Ho seguito il corso lo scorso anno, è cambiato qualche cosa?
- La frequenza è obbligatoria?